1.5 Endliche Automaten

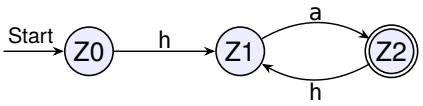
Eine Grammatik *erzeugt* Wörter mit ihren Produktionsregeln, ein **endlicher Automat** *prüft* sie.

Aufgabe 1 *Einstieg: Lachen im Chat*

Chat-Lachen `ha`, `haha`, … ist die Sprache L über $A = \{\mathtt{h}, \mathtt{a}\}$:

```
Lachen = "h" "a" {"h" "a"}.
```

Der Automat rechts *prüft* Eingaben: Er beginnt bei **Start** und folgt für jedes Zeichen dem passenden Pfeil.



a) Verfolge die Eingaben mit dem Finger. Notiere die Zustände. Gehört die Eingabe zu L?

Wort	Zustände	in L?	Wort	Zustände	in L?
`ha`	Z0, Z1, Z2	ja	`haa`	Z0, Z1, Z2, kein Pfeil	nein
`haha`	Z0, Z1, Z2, Z1, Z2	ja	`aha`	Z0, kein Pfeil	nein
`hah`	Z0, Z1, Z2, Z1	nein	ε	Z0	nein

b) Woran erkennt man am Ende, ob eine Eingabe zu L gehört? ganz gelesen und Endzustand

Endlicher Automat

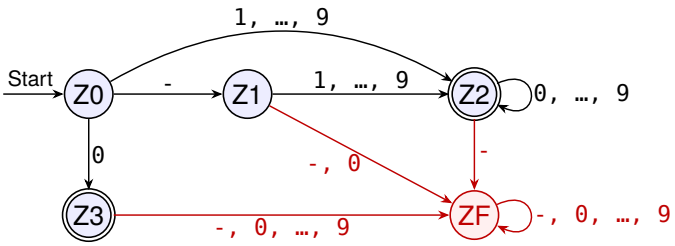
Ein **endlicher** (**erkennender**) **Automat** prüft, ob eine Zeichenkette zu einer Sprache gehört. Er hat ein **Eingabealphabet**, endlich viele **Zustände** (Kreise) mit genau einem **Startzustand** und mindestens einem **Endzustand** (doppelter Kreis) und **Zustandsübergänge** (Pfeile). Jeder Pfeil ist eine einfache **Regel**: *In Z0 wird h gelesen* $\Rightarrow$ *wechsle nach* Z1. Die Zeichnung heißt **Zustandsübergangsdiagramm**, alle Regeln zusammen sind die **Übergangstabelle**.

	h	a
Z0	Z1	–
Z1	–	Z2
Z2	Z1	–

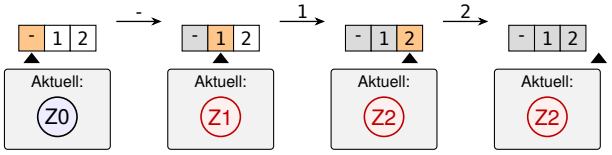
– : kein Übergang

Aufgabe 2 *Der Automat als Maschine: ganze Zahlen*

Ein Compiler prüft, ob in `x = 12-` rechts eine **ganze Zahl** steht. Der Automat erkennt ganze Zahlen ohne führende Nullen über $A = \{\mathtt{-}, \mathtt{0}, \ldots, \mathtt{9}\}$. Als **Maschine** gedacht: Ein **Lesekopf** ▲ liest Zeichen für Zeichen (gelesen grau, aktuell orange). Ungelesene Zeichen bilden die **Restzeichenkette (RZK)**.



a) Ergänze die Anzeigen der Maschine für die Eingabe `-12`. Ergebnis: akzeptiert



RZK leer und Endzustand erreicht

b) Gib die Zustandsfolgen an. Wird die Eingabe akzeptiert? Begründe bei Ablehnung.

Eingabe	`-`	`12-`	`007`
Zustände	Z0, Z1	Z0, Z2, Z2	Z0, Z3
Ergebnis	nein: Z1 ist kein Endzustand	nein: Abbruch in Z2	nein: Abbruch in Z3

So arbeitet jeder Automat: vier einfache Regeln

① Start im Startzustand beim ersten Zeichen.

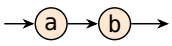
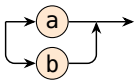
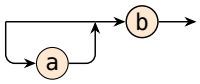
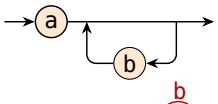
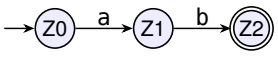
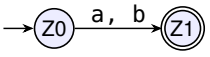
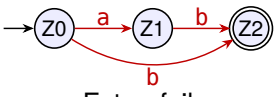
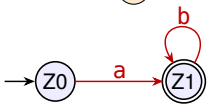
② Passender Pfeil: Wechsel zum Folgezustand. Kein passender Pfeil: Abbruch, abgelehnt.

③ Jeder Übergang verkürzt die RZK um ein Zeichen.

④ **Akzeptiert**, wenn die RZK leer ist und der Automat in einem Endzustand steht.

Aufgabe 3 *Bausteine übersetzen: vom Syntaxdiagramm zum Automaten*

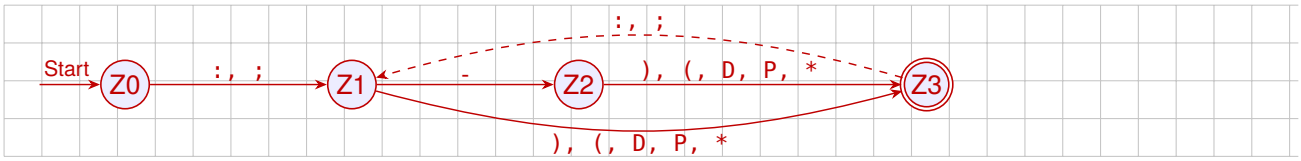
Ein **Zustand** ist eine Stelle im Gleisnetz. Zeichne Option und Wiederholung, ergänze die Regeln.

Aneinanderreihung	**Alternative**	**Option**	**Wiederholung**
`"a" "b"`	`"a" | "b"`	`["a"] "b"`	`"a" {"b"}`
			
			
Pfeile hintereinander	ein Pfeil, alle Zeichen	Extrapfeil überspringt	Schleife am Zustand

Aufgabe 4 *Selbst bauen: Smileys*

a) Übersetze die Smiley-Grammatik aus 1.4 in einen Automaten. Im Automaten stehen nur Zeichen:

```
Smiley = Augen Mitte Mund.    Mitte = ["-"].
Augen = ":" | ";".            Mund = ")" | "(" | "D" | "P" | "*".
```



b) Teste: `:-)` Z0, Z1, Z2, Z3 ✓ `;D` Z0, Z1, Z3 ✓

`:--)` Z0, Z1, Z2, Abbruch ✗

c) Für `Smileyfolge = Smiley {Smiley}.` fehlt ein Pfeil. Ergänze ihn gestrichelt.

Aufgabe 5 *Vollständige Automaten: der Fangzustand*

	-	0	1–9
Z0	Z1	Z3	Z2
Z1	ZF	ZF	Z2
Z2	ZF	Z2	Z2
Z3	ZF	ZF	ZF
ZF	ZF	ZF	ZF

a) Trage die Regeln aus Aufgabe 2 ein (leeres Feld: Abbruch).

b) Ein **Fangzustand** ZF fängt Abbrüche auf: ZF in leere Felder und seine Zeile, dann in Aufgabe 2 einzeichnen.

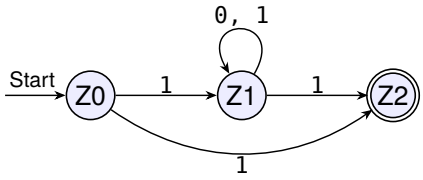
c) Wie endet `12-` jetzt? in ZF ohne Abbruch: abgelehnt

d) Welcher Zustand hat keinen Pfeil nach ZF? Z0

e) (mündlich) Endet eine Ablehnung jetzt immer in ZF?

Aufgabe 6 *Nichtdeterministische Automaten: ungerade Binärzahlen*

Erkennt ungerade Binärzahlen ohne führende Nullen: `1`, `11`, `101`, …



a) Was ist neu? Z0 und Z1 haben für `1` je zwei Pfeile.

b) Für `11` gibt es drei Wege. Gib sie an und entscheide.

Z0, Z2	abgelehnt: RZK nicht leer
Z0, Z1, Z1	abgelehnt: kein Endzustand
Z0, Z1, Z2	RZK leer und Endzustand

c) Wird `11` akzeptiert? ja: Ein erfolgreicher Weg genügt.

Eigenschaften von Automaten

deterministisch: in jedem Zustand für jedes Zeichen höchstens ein Übergang. **nichtdeterministisch**: mehrere Übergänge für ein Zeichen; akzeptiert, wenn mindestens ein Weg bei leerer RZK in einem Endzustand endet. **vollständig**: in jedem Zustand für jedes Zeichen mindestens ein Übergang. Fehlende Übergänge führen in einen **Fangzustand**: kein Endzustand, wird nie verlassen. Ein Automat prüft nur die Syntax.